Skillora: AI-Powered Resume Intelligence & Job Readiness Platform

Agentic AI Project Synopsis
Members:*Palak Bhatt (230839), Manya Agarwal (230811), Vanshik Soni (230887), Harman Singh Kanwar (230656)*

Problem Definition

What problem are you addressing?

Job seekers repeatedly re-tailor their resume for every application, even though their underlying skills, projects and experience stay largely the same. Two problems sit underneath this: (1) project details are scattered across GitHub repos, documents, resumes and notes, so a general AI tool tailoring a resume from chat context alone can omit a component, attach the wrong technology, change a metric, or invent a plausible-sounding but false description; and (2) existing ATS and resume-scoring tools give inconsistent scores for the same resume and job description, and hide the reasoning and detailed breakdown behind paid tiers, leaving candidates without a clear, trustworthy explanation of their score or what to improve.

Who are the intended users?

Job-seeking candidates — particularly students and early-career applicants who apply to multiple roles and need a fast, trustworthy way to tailor their resume per job description and receive transparent, evidence-backed feedback on their job fit.

Objectives & Expected Outcomes

Key Objectives

- Maintain one verified, source-linked profile per candidate (a fact ledger with citations), so every generated resume claim is traceable and tailoring never invents details.
- Automate resume tailoring per job description — analyze the JD, select relevant projects/experience, and write a verified resume — removing the repetitive manual rewriting candidates currently do for each application.
- Provide a transparent, five-component job-fit score with visible evidence per component and concrete, ranked suggestions for improvement, instead of an opaque, paywalled ATS score.

Expected Outcome

A working, local-first multi-agent application where a candidate can add project sources (GitHub, documents, notes), verify the facts extracted from them, paste a job description, and receive a tailored, fact-verified resume along with an explainable job-fit score and fix suggestions — validated against an evaluation set covering fact-extraction accuracy, requirement-matching quality, score stability, and fabrication catch rate.

Proposed Methodology

Agents and their roles

Agent	Framework	Role
Source Extractor	CrewAI	Turns parsed source chunks into candidate facts, each with a quoted source span
JD Analyzer	CrewAI	Turns a job description into structured requirements: must/nice-have, category (skill/experience/education/other), skills, keywords, seniority range

Agent	Framework	Role
Matcher	CrewAI	Links each requirement to ledger facts via exact match, then fuzzy match, then embedding similarity; labels met, partial or missing
Coordinator	Google ADK	Runs the interactive chat session, holds state, delegates to the sub-agents below
Project Selector	Google ADK	Chooses projects/experience for the role by task-relevance (not just tech-stack overlap), with a one-line reason each
Resume Writer	Google ADK	Writes resume bullets and a professional-summary line using fact IDs only
Verifier	Google ADK (callback + tool)	Blocks any claim not supported by its cited source span (deterministic number/technology checks plus an LLM entailment check)
Keyword Coach	Built on Matcher + Writer	For each missing/partial JD keyword, proposes a grounded rewrite if a fact supports it, or surfaces a genuine gap if nothing does; candidate accepts/rejects
Scoring Engine	Plain Python	Computes the deterministic five-component job-fit score, kept as code (not an LLM agent) so it stays consistent across runs

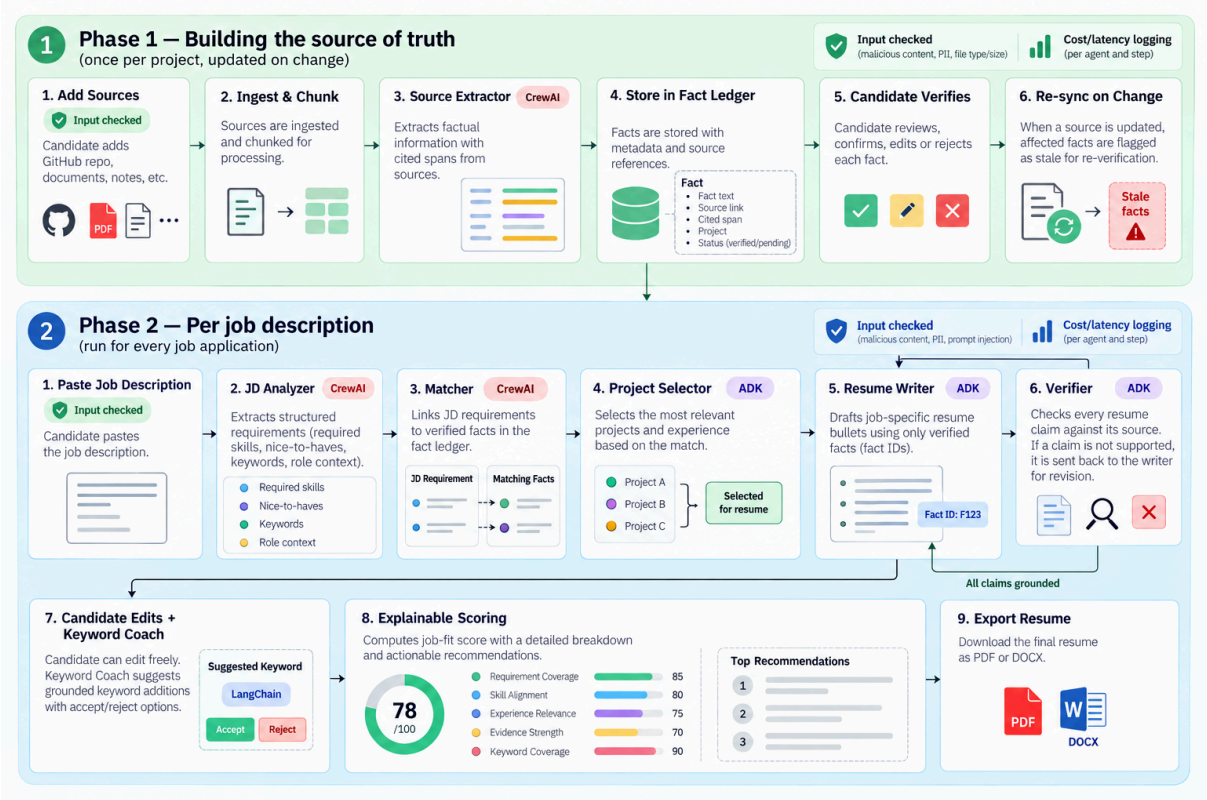
Tools / APIs to be used

- GitHub REST API (repo ingestion)
- pdfplumber, python-docx, python-pptx (document parsing)
- sentence-transformers (local embeddings) and RapidFuzz (fuzzy string matching)
- Jinja + WeasyPrint (PDF export), python-docx (DOCX export)
- FastAPI backend, SQLAlchemy + SQLite (local data layer), Next.js frontend

LLM / model and Agentic AI framework

Google Gemini (free-tier API key) as the underlying LLM, with Ollama as an optional local/private alternative. CrewAI Flows run the deterministic analysis pipeline (ingestion, JD analysis, matching); Google ADK runs the interactive, stateful tailoring side (sessions, sub-agents, callbacks).

Workflow of the proposed system



Feasibility & Resources

Data sources

Candidate-provided material only — public GitHub repos, uploaded documents (PDF/DOCX/PPTX), an existing resume or LinkedIn export, and free-text notes — plus pasted job descriptions. No external dataset is required.

APIs / tools

GitHub REST API, Google Gemini API (free tier), sentence-transformers, RapidFuzz, pdfplumber, python-docx, python-pptx, Jinja + WeasyPrint

Software / frameworks

FastAPI, SQLAlchemy + SQLite, Next.js, CrewAI, Google ADK, Docker Compose

Compute / resources required

Runs entirely on a laptop for the local build (SQLite plus local embeddings); one free Gemini API key per team member is sufficient. A later, optional deployment stage moves to hosted Postgres with pgvector and a small cloud host (Cloud Run, Vercel, or Hugging Face Spaces as fallback) — this is a packaging step reusing the same code, not a redesign.

Team Members & Individual Understanding

Member	Role / Responsibility	Individual Understanding
Palak Bhatt	Resume Tailoring Engine + Keyword Coach (Google ADK), plus the entire frontend (Next.js)	Owns the Resume Writer and Keyword Coach agents (Google ADK) plus the entire Next.js frontend across all 7 screens. The Resume Writer drafts resume bullets and a summary line using fact IDs only, so nothing is invented;

Member	Role / Responsibility	Individual Understanding
		the Keyword Coach proposes grounded rewrites for missing JD keywords, or flags a genuine gap if no fact supports it. Also owns cross-team integration and the eval runner, plus the fabrication catch rate documentation.
Manya Agarwal	Profile Builder — parsers, Source Extractor agent, fact ledger, verification API (CrewAI)	Owns the Source Extractor agent (CrewAI) and fact ledger — parsing GitHub repos, documents, and notes into candidate facts, each tied to a quoted source span, plus the candidate verification API where facts are confirmed, edited, or rejected. Also responsible for the extraction accuracy metric in evaluation.
Vanshik Soni	Job Description Analyzer & Matching — JD Analyzer and Matcher agents (CrewAI)	the JD Analyzer turns a pasted job description into structured requirements (must/nice-have, category, skills, seniority), and the Matcher links each requirement to fact-ledger entries via exact → fuzzy → embedding-similarity matching, labeling them met/partial/missing. Also responsible for the requirement-matching quality metric in evaluation.
Harman Singh Kanwar	Explainable Scoring & Export — scoring engine and PDF/DOCX export (plain Python, light CrewAI)	deterministic Python turns Matcher's requirement labels into a transparent job-fit score with suggestions, plus PDF/DOCX export. Also owns the fabrication catch rate metric, tied to the Verifier's grounding checks.

Project Timeline

Activity	Week(s)	Deliverable
Problem & Design	September, 3rd week	Phase-1 submission (10%): one-page Project Charter/Synopsis, Project Timeline, Team Responsibility Matrix; fact-ledger schema agreed; CrewAI + ADK environment set up

Activity	Week(s)	Deliverable
Development / Coding	Late Sept – October, 3rd/4th week	Ingestion and fact extraction, JD Analyzer, Matcher and the score-engine core built and the Phase-2 working prototype/demo (30%, Oct 3rd/4th week) with a Progress Report
Integration & Testing / Debugging	November, 1st/2nd week	Resume Writer and Verifier built; edit/re-score loop; all four features connected into one end-to-end prototype
Evaluation & Refinement	November, 2nd/3rd week	Evaluation set run and recorded: extraction accuracy, requirement-matching quality, score stability, fabrication catch rate
Presentation / Interface UI	November, 2nd/3rd week	All 7 frontend screens working against the real backend, polished for demo
Deployment	November, 3rd week	Containerised with Docker Compose, sign-in and hosted Postgres added, deployed with a public URL
Final Demo	November, last week	Everything finished and rehearsed internally: final functional application, GitHub repo, final project report (course template), final presentation PPT & demonstration, final group/self contribution matrix — ready ahead of the handout's official Phase-3 evaluation (40%) in the last week of the course / early December, followed by Viva

Team Responsibility Matrix

Activity	Manya Agarwal (Profile Builder)	Vanshik Soni (JD Analyzer)	Harman (Scoring & Export)	Palak (Tailoring + Frontend)
Agent Design	✓	✓	✓ (scoring logic design)	✓
Agent / Tool Development	✓	✓	✓	✓
Integration & Testing	✓ (own feature)	✓ (own feature)	✓ (own feature)	✓ (own feature + cross-team integration, owns eval runner)
Evaluation & Documentation	✓ (extraction accuracy)	✓ (matching quality)	✓ (score stability)	✓ (fabrication catch rate, keeps docs/ in sync)
Presentation / Interface UI	✓ (own feature's screens: Verify Facts, Add Sources)	✓ (own feature's screens: New Job)	✓ (own feature's screens: Score	✓ (own feature's screens: Tailor & Edit, Keyword

Activity	Manya Agarwal (Profile Builder)	Vanshik Soni (JD Analyzer)	Harman (Scoring & Export)	Palak (Tailoring + Frontend)
			Report, Export)	Coach panel, chat sidebar)
Deployment				✓ (owns shared docker-compose.yml review)